

Document Number: P2628R0

Date: 2022-07-01

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>

Authors: Gonzalo Brito Gadeschi

Audience: Concurrency

Extend `barrier` APIs with `memory_order`

- Extend `barrier` APIs with `memory_order`
 - Motivation
 - Tony Tables
 - Wording
 - Compatibility with P2535

Motivation

`std::barrier` is a synchronization primitive that orders memory visibility across threads. Its operations - `arrive`, `wait`, `arrive_and_wait`, `arrive_and_drop` - guarantee visibility of **all** memory operations performed before the `arrive` to all threads that are unblocked from the `wait` (after they are unblocked from it).

Sometimes, guaranteeing the memory visibility of **all** memory operations is not required nor desired.

Examples:

1. Communicating “outside” the C++ abstract machine. Examples:

1. Every thread participating in the barrier opens, writes to, and closes a file. Threads use a barrier to synchronize whether all files have been closed using `bar.arrive_and_wait(1, memory_order_relaxed)`. The memory visibility is ensured by the filesystem outside the C++ abstract machine. This is similar to how, e.g., `MPI_Ibarrier` does not establish cumulativity across MPI ranks for memory operations on `MPI_Windows`.
2. Every thread participating in the barrier is responsible for configuring a part of a machine via `volatile` operations. After the machine is configured, one thread should start it before releasing any threads from the barrier. Threads achieve this by using `memory_order_relaxed` `arrive/wait` operations together with a `CompletionFunction` that is run after the last thread arrived.

2. **Object fences:** P2535 (<https://wg21.link/P2535>) - and P0153 (<https://wg21.link/p0153>) before it - proposes `atomic_object_fence` and `atomic_message_fence`. These fences only apply to a sub-set of all objects and memory operations. This paper enables applications to compose `std::barrier` with P2535 (<https://wg21.link/P2535>) fences. This is explored in its own section further below.

Other synchronization primitives could be extended in an analogous way as well, but we choose to focus on `std::barrier` during the initial revisions of this paper. Some synchronization primitives like `std::atomic::wait` already expose a `memory_order` parameter.

Tony Tables

Before	After
<pre>// Thread 0: x = 1; atomic_object_fence(memory_order_release, x); bar.arrive(); // release fence // Thread 1 bar.arrive_and_wait(); // acquire fence atomic_object_fence(memory_order_acquire, x); assert(x == 1);</pre>	<pre>// Thread 0: x = 1; atomic_object_fence(memory_order_release, x); bar.arrive(1, <u>memory_order_relaxed</u>); // no fence // Thread 1 bar.arrive_and_wait(<u>memory_order_relaxed</u>); // no fence atomic_object_fence(memory_order_acquire, x); assert(x == 1);</pre>

Before: does not benefit from `atomic_object_fence` since the barrier operations insert full-memory fences.

After: benefits from `atomic_object_fence` since the barrier inserts no fences.

Wording

Note: an implementation is available here (<https://godbolt.org/z/b4WcjhEnP>) in the `barrier_memory_order.hpp` file.

`thread.barrier.class` (<http://eel.is/c++draft/thread.barrier.class>):

```

namespace std {
    template<class CompletionFunction = see below>
    class barrier {
    public:
        using arrival_token = see below;

        static constexpr ptrdiff_t max() noexcept;

        constexpr explicit barrier(ptrdiff_t expected,
                                   CompletionFunction f = CompletionFunction());
        ~barrier();

        barrier(const barrier&) = delete;
        barrier& operator=(const barrier&) = delete;

        [[nodiscard]] arrival_token arrive(ptrdiff_t update = 1, memory_order = memory_order_acquire) const;
        void wait(arrival_token&& arrival, memory_order = memory_order_acquire) const;

        void arrive_and_wait(ptrdiff_t update = 1, memory_order = memory_order_acq_rel);
        void arrive_and_drop(ptrdiff_t update = 1, memory_order = memory_order_release);

    private:
        CompletionFunction completion;      // exposition only
    };
}

```

Unresolved question: should we add `update = 1` parameters to the APIs that lack them? This revision does that only to show how that would look like.

1. Each barrier phase consists of the following steps:
 1. The expected count is decremented by each call to `arrive` or `arrive_and_drop`.
 2. When the expected count reaches zero, the phase completion step is run. For the specialization with the default value of the `completionFunction` template parameter, the completion step is run as part of the call to `arrive` or `arrive_and_drop` that caused the expected count to reach zero. For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.
 3. When the completion step finishes, the expected count is reset to what was specified by the expected argument to the constructor, possibly adjusted by calls to `arrive_and_drop`, and the next phase starts.
2. Each phase defines a phase synchronization point. Threads that arrive at the barrier during the phase can block on the phase synchronization point by calling `wait`, and will remain blocked until the phase completion step is run.

3. The phase completion step that is executed at the end of each phase has the following effects:

1. Invokes the completion function, equivalent to `completion()` .
2. Unblocks all threads that are blocked on the phase synchronization point.

UNRESOLVED QUESTION: do we need to change something else around here or does the “as if” below in “4.” suffice?

The end of the completion step strongly happens before the returns from all calls that were unblocked by the completion step. For specializations that do not have the default value of the `CompletionFunction` template parameter, the behavior is undefined if any of the barrier object’s member functions other than `wait` are called while the completion step is in progress.

4. Concurrent invocations of the member functions of barrier, other than its destructor, do not introduce data races as if they were atomic operations performed with the `memory_order` associated with them. The member functions `arrive` and `arrive_and_drop` execute atomically.

5. `CompletionFunction` shall meet the `Cpp17MoveConstructible` (Table 30) and `Cpp17Destructible` (Table 34) requirements.

`is_nothrow_invocable_v<CompletionFunction&>` shall be true.

6. The default value of the `CompletionFunction` template parameter is an unspecified type, such that, in addition to satisfying the requirements of `CompletionFunction` , it meets the `Cpp17DefaultConstructible` requirements (Table 29) and `completion()` has no effects.

7. `barrier::arrival_token` is an unspecified type, such that it meets the `Cpp17MoveConstructible` (Table 30), `Cpp17MoveAssignable` (Table 32), and `Cpp17Destructible` (Table 34) requirements.

```
static constexpr ptrdiff_t max() noexcept;
```

- Returns: The maximum expected count that the implementation supports.

```
constexpr explicit barrier(ptrdiff_t expected,
                           CompletionFunction f = CompletionFunction());
```

- Preconditions: `expected >= 0` is true and `expected <= max()` is true.
- Effects: Sets both the initial expected count for each barrier phase and the current expected count for the first phase to `expected`. Initializes completion with `std::move(f)`. Starts the first phase.

[Note 1: If `expected` is 0 this object can only be destroyed. — end note]

- Throws: Any exception thrown by `CompletionFunction`'s move constructor.

```
[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1, memory_order order = memo
```



- Preconditions: `update > 0` is true, ~~and~~ `update` is less than or equal to the expected count for the current barrier phase, and order is `memory_order_relaxed` or `memory_order_release`.
- Effects: Constructs an object of type `arrival_token` that is associated with the phase synchronization point for the current phase. Then, decrements the expected count by `update`.
- Synchronization: The call to `arrive` strongly happens before the start of the phase completion step for the current phase.
- Returns: The constructed `arrival_token` object.
- Throws: `system_error` when an exception is required (`[thread.req.exception]`).
- Error conditions: Any of the error conditions allowed for mutex types (`[thread.mutex.requirements.mutex]`).

[Note 2: This call can cause the completion step for the current phase to start. — end note]

```
void wait(arrival_token&& arrival, memory_order order = memory_order_acquire) cons
```

- Preconditions: arrival is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object, and order is `memory_order_relaxed` or `memory_order_acquire`.
- Effects: Blocks at the synchronization point associated with `std::move(arrival)` until the phase completion step of the synchronization point's phase is run.

[Note 3: If arrival is associated with the synchronization point for a previous phase, the call returns immediately. — end note]

- Throws: `system_error` when an exception is required ([thread.req.exception]).
- Error conditions: Any of the error conditions allowed for mutex types ([thread.mutex.requirements.mutex]).

```
void arrive_and_wait(ptrdiff_t update = 1, memory_order order = memory_order_acq_r
```

- Preconditions: order is `memory_order_relaxed` or `memory_order_acq_rel`.
- Effects: Equivalent to:

```
order_arrive = order == memory_order_acq_rel? memory_order_release : order;  
order_wait = order == memory_order_acq_rel? memory_order_acquire : order;  
wait(arrive(update, order_arrive), order_wait);
```

.

```
void arrive_and_drop(ptrdiff_t update = 1, memory_order order = memory_order_relea
```

- Preconditions: The expected count for the current barrier phase is greater than zero, `update > 0` is true, `update` is less than or equal to the expected count for the current barrier phase, and order is `memory_order_relaxed` or `memory_order_release`.

Rationale for using `update` for initial and current phase counts: safety, this prevents the initial count from going under the current count accidentally.

- Effects: Decrements the initial expected count for all subsequent phases by `oneupdate`. Then decrements the expected count for the current phase by `oneupdate`.
- Synchronization: The call to `arrive_and_drop` strongly happens before the start of the phase completion step for the current phase.
- Throws: `system_error` when an exception is required ([`thread.req.exception`]).
- Error conditions: Any of the error conditions allowed for mutex types ([`thread.mutex.requirements.mutex`]).

[Note 4: This call can cause the completion step for the current phase to start. — end note]

Compatibility with P2535 (<https://wg21.link/p2535>)

P2535 (<https://wg21.link/P2535>) (and P0153 (<https://wg21.link/p0153>) before it) propose extending C++ with object fences. If we were to add object fences to C++, it could make sense to further extend `barrier` APIs to support them, e.g., as follows:

```
int& data;
data = 42;
bar_x.arrive(1, obj_fence, data);
bar_y.arrive(1, obj_fence, data);
```

These APIs would be semantically similar - although not identical - to the following code using the APIs in this proposal.

```
int& data;
data = 42;
atomic_object_fence(memory_order_release, data);
bar_x.arrive(1, memory_order_relaxed);
atomic_object_fence(memory_order_release, data);
bar_y.arrive(1, memory_order_relaxed);
```

Combining both APIs is still valuable since it allows power users to write code with less fences where necessary:

```
int& data;  
data = 42;  
atomic_object_fence(memory_order_release, data);  
bar_x.arrive(1, memory_order_relaxed);  
bar_y.arrive(1, memory_order_relaxed);
```